J
2

Session 1 · Jour 2 / 9

Producer API Java · Consumer API Python

FORMATION APACHE KAFKA · 9 JOURS

Producer API Java & Consumer API Python

Producer API Java

KafkaProducer · acks · batching · idempotence

Consumer API Python

kafka-python · poll · commit · error handling

Sérialisation JSON & Avro

StringSerializer · JsonSerializer · Schema Registry

Consumer Groups avancés

Seek · assign · rebalance listeners · pause/resume

⏱ 1 jour · 40% théorie / 60% TP · 50 slides | Prérequis : Jour 1 validé · Java 17 · Python 3.9+ · Maven

Plan du Jour 2 — Programme complet (50 slides)

01 — 9h00–10h30
Producer API Java — Configuration et patterns avancés — Slides 3–12

02 — 10h45–12h00
Consumer API Python — Poll loop et gestion des offsets — Slides 13–22

03 — 13h00–14h30
Sérialisation — JSON, Avro et Schema Registry — Slides 23–32

04 — 14h30–15h30
Consumer Groups avancés — Seek, Assign, Pause/Resume — Slides 33–40

05 — 15h45–17h30
TP — Fil rouge : Pipeline e-commerce Java + Python — Slides 41–50

Pause café 10h30–10h45 · Déjeuner 12h00–13h00 · Pause 15h30–15h45 · Fin 17h30

01

Producer API Java

KafkaProducer · Configuration · Idempotence · Transactions · Patterns avancés

Architecture KafkaProducer — Vue d'ensemble

☕ KafkaProducer<K,V>

Le KafkaProducer est thread-safe et réutilisable. Il encapsule le réseau, la sérialisation, le partitionnement et le batching. Une seule instance par application — ne pas en créer une par message !

KafkaProducer — Setup complet production

```
import org.apache.kafka.clients.producer.*;
import java.util.Properties;

public class OrderProducer {
  public static void main(String[] args) throws Exception {
    Properties props = new Properties();

    // ─── Connexion ──────────────────────────────────────────────
    props.put("bootstrap.servers", "broker1:9092,broker2:9092");

    // ─── Sérialisation ──────────────────────────────────────────
    props.put("key.serializer",
      "org.apache.kafka.common.serialization.StringSerializer");
    props.put("value.serializer",
      "org.apache.kafka.common.serialization.StringSerializer");

    // ─── Durabilité ─────────────────────────────────────────────
    props.put("acks", "all");
    props.put("enable.idempotence", "true");
    props.put("retries", Integer.MAX_VALUE);
    props.put("max.in.flight.requests.per.connection", "5");

    // ─── Performance ────────────────────────────────────────────
```

Paramètres essentiels

`acks`	all → durabilité max
`enable.idempotence`	true → exactly-once
`retries`	MAX_INT (avec idempotence)
`batch.size`	65536 (64KB)
`linger.ms`	5ms → grouper msgs
`compression.type`	snappy · lz4 · zstd
`buffer.memory`	32MB ring buffer
`request.timeout.ms`	30000 (30s)

Modes d'envoi Producer — Sync · Async · Fire & Forget

Fire & Forget

`producer.send(record)`

Envoie et n'attend pas. Aucune gestion d'erreur. Le callback n'est pas fourni. Utilisé pour des logs non-critiques où la perte est acceptable.

```
producer.send(new ProducerRecord<>(
  "logs", "server-1", "Page not found"));
// Pas de callback — on ignore le résultat
// Perte possible si broker crash
```

Asynchrone (recommandé)

`producer.send(record, callback)`

Envoie avec un callback invoqué quand le broker confirme (ou échoue). Non-bloquant. Pattern optimal pour la production : bon débit ET gestion des erreurs.

```
producer.send(record, (metadata, exception) ->
{
  if (exception != null) {
    log.error("Erreur envoi: {}",
exception.getMessage());
    // retry / DLQ selon la politique
  } else {
    log.info("Sent to partition={} offset={}",
      metadata.partition(), metadata.offset());
  }
});
```

Synchrone (bloquant)

`producer.send(record).get()`

Bloque jusqu'à la confirmation du broker. Garantit l'envoi avant de continuer. Faible débit mais ordre parfaitement contrôlé. Utile pour les migrations ou les tests.

```
try {
  RecordMetadata meta =
    producer.send(record).get(30,
TimeUnit.SECONDS);
  log.info("Confirmé: offset={}",
meta.offset());
} catch (ExecutionException e) {
  // L'envoi a échoué après tous les retries
  throw new RuntimeException("Échec définitif",
e);
}
```

Pattern production : send(record, callback) — non-bloquant + gestion d'erreurs + DLQ pour les messages en échec.

Idempotence & Transactions — Exactly-Once Semantics

🔒 Exactly-Once Semantics (EOS)

Par défaut Kafka offre at-least-once : un message peut être dupliqué en cas de retry. L'idempotence garantit exactly-once au niveau producer. Les transactions permettent exactly-once atomique multi-topics.

Idempotence Producer — enable.idempotence=true

Comment ça marche :

- ✔ Chaque producer reçoit un PID (Producer ID) unique du broker
- ✔ Chaque message a un numéro de séquence (0, 1, 2...)
- ✔ Le broker détecte et ignore les doublons (même PID + même seq)
- ✔ Retries transparents sans duplication · max.in.flight ≤ 5

Config minimale idempotence

```
props.put("enable.idempotence", "true");
// Active automatiquement :
// acks=all · retries=MAX · max.in.flight=5
// Garantie : pas de doublon broker-side
```

Transactions — Atomique multi-topics

Cas d'usage : écrire sur 2 topics en même temps
→ Les deux réussissent OU les deux sont annulés

Transactions Java

```
props.put("transactional.id","order-producer-1");
KafkaProducer<String,String> p=new KafkaProducer<>(props);
p.initTransactions();

try {
  p.beginTransaction();

  // Écriture atomique sur 2 topics
  p.send(new ProducerRecord<>("orders", key, val));
  p.send(new ProducerRecord<>("audit.log", key, log));

  p.commitTransaction(); // Les 2 ou aucun
} catch (ProducerFencedException e) {
  p.close(); // Autre instance active
} catch (KafkaException e) {
  p.abortTransaction(); // Rollback
}
```

Transactions requis : isolation.level=read_committed côté consumer pour ne lire que les messages commités.

Partitionnement avancé — Stratégies et Custom Partitioner

🗂️ Stratégies de partitionnement

Le choix de la clé de partition est critique : il détermine l'ordre des messages, la répartition de charge et le parallélisme des consumers. Une mauvaise clé crée des hot partitions et des déséquilibres.

DefaultPartitioner (par clé)

murmur2(key) % numPartitions

```
// Message avec clé string
ProducerRecord<String,String> r = new ProducerRecord<>(
  "commerce.orders",
  "order-1",  // Clé = ID commande
  "{"id":1,"status":"CREATED"}"
);
// order-1 → toujours partition 2
// order-1 PAID → aussi partition 2 → ordre garanti !
```

Avantage : ordre garanti par entité · Même clé = même partition · Idéal pour les entités métier

Custom Partitioner

Logique métier personnalisée

```
public class PriorityPartitioner implements Partitioner {
  @Override
  public int partition(String topic, Object key,
      byte[] keyBytes, Object value, byte[] valueBytes,
      Cluster cluster) {
    String orderJson = (String) value;
    boolean isVIP = orderJson.contains(""tier":"vip"");
    int numParts = cluster.partitionCountForTopic(topic);
    // VIP → partition 0 toujours (traitement prioritaire)
    return isVIP ? 0 : (Math.abs(key.hashCode()) %
(numParts-1))+1;
  }
}
// Config :
props.put("partitioner.class","com.acme.PriorityPartitioner");
```

Cas d'usage : priorité VIP · Partitionnement géographique · Routing par tenant (SaaS multi-tenant)

Hot partition : si une clé représente 80% du trafic, sa partition sera surchargée — préférer des clés à haute cardinalité.

Gestion des erreurs Producer — Retry, DLQ et Circuit Breaker

Classification des erreurs Kafka Producer

Erreurs retriables

NotLeaderForPartitionException, NetworkException, TimeoutException. Kafka retente automatiquement si retries > 0. Transparent pour l'application.

Erreurs non-retriables

RecordTooLargeException, SerializationException, InvalidTopicException. Échec permanent — envoyer en DLQ.

Erreurs d'expiration

delivery.timeout.ms dépassé sans confirmation. Tous les retries épuisés. Décision : DLQ ou alarme critique.

Callback avec DLQ — Pattern production complet

```
producer.send(record, (metadata, exception) -> {
  if (exception != null) {
    if (exception instanceof RecordTooLargeException
        || exception instanceof SerializationException) {
      // Erreur permanente → Dead Letter Queue
      ProducerRecord<String,String> dlqRecord =
        new ProducerRecord<>("commerce.orders.dlq",
          record.key(), record.value());
      dlqProducer.send(dlqRecord);
      metrics.increment("kafka.producer.dlq");
    } else {
```

Paramètres retry

retries	MAX_INT (avec idempotence)
delivery.timeout.ms	120000 (2 min)
retry.backoff.ms	100 (100ms entre retries)
request.timeout.ms	30000 (30s)

Dead Letter Queue (DLQ) : topic séparé pour les messages en échec permanent · analyser et rejouer manuellement.

Compression et tuning performance Producer

Mode d'envoi	Fire & Forget	Asynchrone (recommandé)	Synchrone (bloquant)
API Java	**producer.send(record)**	**producer.send(record, callback)**	**producer.send(record).get()**
Blocking ?	Non — ignore erreurs	**Non — callback async**	Oui — bloque le thread
Gestion erreurs	Aucune — perte possible	**Via callback (retry/DLQ)**	Via try/catch
Débit	Maximum	**Très élevé**	Faible (bloquant)
Usage	Logs non-critiques	⭐ **Standard production**	Tests · Migrations · Audit

ProducerInterceptor & Headers Kafka — Observabilité

🔍 ProducerInterceptor

Les interceptors permettent d'injecter du comportement transversal : tracing distribué, métriques, corrélation IDs, logging structuré — sans modifier le code métier. Chaînable par ordre de configuration.

ProducerInterceptor — Tracing et métriques

```
public class TracingInterceptor<K,V>
    implements ProducerInterceptor<K,V> {

  @Override
  public ProducerRecord<K,V> onSend(ProducerRecord<K,V> record) {
    // Injecter correlation ID dans les headers
    String correlationId = UUID.randomUUID().toString();
    record.headers()
      .add("X-Correlation-Id",
           correlationId.getBytes(StandardCharsets.UTF_8))
      .add("X-Sent-At",
           Instant.now().toString().getBytes());
    // Démarrer le span de tracing (OpenTelemetry)
    tracer.inject(record.headers());
    return record;
```

Config interceptor

```
props.put("interceptor.classes",
  "com.acme.TracingInterceptor," +
  "com.acme.MetricsInterceptor");
```

Headers Kafka

Les headers sont des métadonnées clé-valeur attachées à chaque message. Ils traversent tout le pipeline sans modifier la value.

Headers Java

```
// Écriture header
record.headers().add(
  "source-service",
  "order-api".getBytes());

// Lecture côté consumer
record.headers().forEach(h -> {
  String key = h.key();
  String val = new String(h.value());
```

X-Correlation-Id	Tracing distribué
X-Sent-At	Timestamp émetteur
X-Schema-Version	Version schéma
X-Source-Service	Service émetteur

OpenTelemetry + Kafka : injection du traceId dans les headers → observabilité end-to-end dans les microservices.

Benchmark Producer — kafka-producer-perf-test

⚡ **kafka-producer-perf-test.sh**

Outil CLI intégré pour mesurer le débit réel du producer dans votre environnement. Paramétrable : nombre de messages, taille, débit cible, configuration de compression.

Scénarios de benchmark

```
# Benchmark débit maximal (acks=1, snappy)
kafka-producer-perf-test.sh \
  --topic commerce.orders \
  --num-records 1000000 \
  --record-size 1024 \
  --throughput -1 \
  --producer-props bootstrap.servers=localhost:19092 \
    acks=1 compression.type=snappy \
    batch.size=65536 linger.ms=5

# Résultat attendu :
# 1000000 records sent, 450000.0 records/sec (440.43 MB/sec)
# avg latency 5.23 ms, 99th % 18.0 ms

# Benchmark durabilité (acks=all)
```

Interpréter les résultats

```
# Colonnes clés :
# records/sec   → débit (objectif > 100k/sec)
# MB/sec        → bande passante utilisée
# avg latency   → latence moyenne (ms)
# 99th % lat    → latence P99 (objectif < 50ms)
```

Résultats types (i3.4xlarge)

`acks=0, no compress`
800k msg/s, 0.5ms

`acks=0, snappy`
600k msg/s, 0.6ms

`acks=1, snappy`
400k msg/s, 2ms

`acks=all, snappy`
180k msg/s, 8ms

`acks=all, idempotent`
150k msg/s, 10ms

✔ **Standard prod :**
acks=all + snappy → ~150-180k msg/s

Quiz — Section 01 : Producer API Java

❓ enable.idempotence=true garantit :

- ○ Messages chiffrés automatiquement
- ✔ **Pas de doublons même avec les retries — exactly-once producer-side**
- ○ Transactions atomiques multi-topics
- ○ Latence inférieure à 1ms

❓ Quel mode d'envoi est recommandé en production ?

- ○ producer.send(record) sans callback
- ○ producer.send(record).get() — synchrone bloquant
- ✔ **producer.send(record, callback) — async + gestion erreurs**
- ○ producer.sendAsync(record)

❓ linger.ms=5 + batch.size=65536 signifie :

- ○ Messages envoyés toutes les 5 secondes
- ✔ **Attendre 5ms pour remplir un batch de 64KB avant envoi**
- ○ Max 65536 messages en mémoire
- ○ Compression activée automatiquement

❓ Quel algorithme de compression est recommandé en prod ?

- ○ gzip — meilleure compression
- ○ none — moins de CPU
- ✔ **snappy ou zstd — meilleur compromis**
- ○ lz4 uniquement pour Kafka

02

Consumer API Python

kafka-python · Poll loop · Commit manuel · Error handling · Patterns avancés

KafkaConsumer Python — Setup et configuration

🐍 kafka-python — KafkaConsumer

La bibliothèque kafka-python est l'implémentation Python officielle du client Kafka. Elle expose une API similaire au client Java. Installer : pip install kafka-python==2.0.2

consumer_setup.py — Configuration complète

```
from kafka import KafkaConsumer
from kafka.errors import KafkaError
import json, logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ─── Configuration consumer ──────────────────────────────────────
consumer = KafkaConsumer(
    'commerce.orders',          # topic(s) à consommer
    bootstrap_servers=['broker1:9092','broker2:9092'],
    group_id='orders-processor-v1',
    auto_offset_reset='earliest',  # début si pas d'offset
    enable_auto_commit=False,      # commit manuel OBLIGATOIRE
    max_poll_records=500,
    session_timeout_ms=30000,      # 30s avant considéré mort
    heartbeat_interval_ms=10000,   # ping toutes les 10s
    max_poll_interval_ms=300000,   # 5min max entre polls
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),
    key_deserializer=lambda m: m.decode('utf-8') if m else None,
    # Sécurité (production)
    # security_protocol='SASL_SSL',
    # sasl_mechanism='PLAIN',
    # sasl_plain_username='user'
```

Paramètres clés Python

group_id	Identifiant du Consumer Group
auto_offset_reset	earliest · latest · none
enable_auto_commit	False en prod (commit manuel)
max_poll_records	500 msgs par poll
session_timeout_ms	30000 (30s heartbeat)
max_poll_interval_ms	300000 (5min max)
value_deserializer	Fonction de désérialisation
consumer_timeout_ms	Timeout si vide (-1=infini)

pip install kafka-python==2.0.2 · Ou confluent-kafka-python (wrapper librdkafka — plus performant pour prod).

Poll Loop Python — Pattern complet avec commit manuel

orders_consumer.py — Poll loop production

```
from kafka import KafkaConsumer, TopicPartition
from kafka.errors import CommitFailedError
import json, signal, sys

running = True
def shutdown(sig, frame):
    global running
    running = False
    logger.info("Arrêt demandé...")
signal.signal(signal.SIGTERM, shutdown)
signal.signal(signal.SIGINT, shutdown)

try:
  while running:
    # poll() retourne un dict {TopicPartition: [records]}
    msg_pack = consumer.poll(timeout_ms=1000, max_records=500)

    if not msg_pack:
      continue  # Pas de messages — retry

    for tp, messages in msg_pack.items():
      for message in messages:
        try:
          # ─── Traitement métier ──────────────────────────────
          order = message.value  # déjà désérialisé (json)
          logger.info(f"Processing order {order['id']} "
                      f"from partition={message.partition} "
                      f"offset={message.offset}")
          process_order(order)   # votre logique métier
```

Points clés du poll loop

`timeout_ms=1000`
Attend 1s si pas de message (évite busy-wait)

`max_records=500`
Limite le batch pour éviter OOM

`commit() APRÈS traitement`
at-least-once garanti

`try/except par message`
Isolation des erreurs unitaires

`send_to_dlq()`
Messages KO → topic .dlq

`signal.SIGTERM`
Arrêt gracieux du consumer

`consumer.close()`
Libère les partitions immédiatement

consumer.close() est critique : déclenche le rebalancing immédiat et évite d'attendre session_timeout_ms.

Stratégies de commit offset — At-least-once vs At-most-once

📍 Commit d'offset en Python

Le commit est la sauvegarde de la position de lecture du consumer. enable_auto_commit=False est obligatoire en production pour garantir at-least-once processing.

❌ Auto-commit (dangereux)

`enable_auto_commit=True · interval=5s`

```
# DANGER : commit avant traitement
consumer = KafkaConsumer('orders',
  enable_auto_commit=True,
  auto_commit_interval_ms=5000)

for message in consumer:
  # Si crash ici AVANT traitement
  # et que l'offset a été commité
  # → message PERDU (at-most-once !)
  process_order(message.value)
```

Risque : commit avant processing → perte de messages si crash pendant le traitement

✔ Commit sync (recommandé)

`enable_auto_commit=False · commitSync`

```
# CORRECT : commit APRÈS traitement
for msg_pack in consumer:
  for tp, messages in msg_pack.items():
    for message in messages:
      process_order(message.value)
  # Commit après tout le batch traité
  consumer.commit()
  # En cas de crash avant commit :
  # les messages seront retraités
  # (at-least-once → idempotent requis)
```

Garantie : at-least-once. Messages retraités si crash avant commit. Rendre le processing idempotent.

⚡ Commit par offset précis

`commitSync(offsets={tp: OffsetAndMetadata})`

```
from kafka import OffsetAndMetadata,
TopicPartition

# Commit offset précis après chaque
message
for message in consumer:
  process_order(message.value)
  tp = TopicPartition(message.topic,

message.partition)
  # Commit offset+1 (prochain à lire)
  consumer.commit({
    tp:
OffsetAndMetadata(message.offset+1)
  })
  # Granularité maximale : 1
commit/message
```

Contrôle fin : commit à l'offset exact. Overhead élevé. Utile pour processing très long par message.

Règle : always commit AFTER processing. Si idempotent : at-least-once est safe. Sinon : exactly-once via transactions.

Gestion des erreurs Consumer — Retry, DLQ et alertes

Architecture de gestion d'erreurs robuste

Error handling + DLQ Python

```python
from kafka import KafkaProducer
import json, traceback

dlq_producer = KafkaProducer(
    bootstrap_servers=['broker1:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

def process_with_error_handling(message):
    try:
        order = message.value
        # Validation métier
        if 'id' not in order or 'status' not in order:
            raise ValueError(f"Schéma invalide: {order}")

        result = process_order(order)
        logger.info(f"✔ Order {order['id']} processed")
        return True

    except ValueError as e:
```

Stratégie DLQ complète

Topic DLQ	commerce.orders.dlq
Contenu	Message original + erreur + metadata
Rétention DLQ	30 jours minimum
Monitoring	Alerter si DLQ non vide
Replay	Script de rejeu après correction
Analyse	Identifier les erreurs récurrentes

Replay depuis la DLQ

```python
# Relire la DLQ et rejouer les messages corrigés
dlq_consumer = KafkaConsumer('commerce.orders.dlq', group_id='dlq-replayer', ...)
for msg_pack in dlq_consumer:
    for tp, messages in msg_pack.items():
        logger.warning(f"Erreur DB transitoire: {e}")
```

Monitorer la taille de la DLQ avec kafka-consumer-groups.sh : LAG > 0 sur commerce.orders.dlq = alarme.

ConsumerRebalanceListener — Gérer le rebalancing en Python

🔄 ConsumerRebalanceListener

Le RebalanceListener permet d'exécuter du code AVANT que les partitions soient révoquées (commit des offsets en cours) et APRÈS leur assignation (initialisation, warmup). Critique pour éviter la perte de données.

rebalance_listener.py — Pattern complet

```
from kafka import ConsumerRebalanceListener, TopicPartition

class OrdersRebalanceListener(ConsumerRebalanceListener):

    def __init__(self, consumer, offset_manager):
        self.consumer = consumer
        self.offset_manager = offset_manager

    def on_partitions_revoked(self, revoked):
        """Appelé AVANT que les partitions soient redistribuées.
           Contexte : rebalancing en cours — commiter les offsets !"""
        logger.info(f"Partitions révoquées: {revoked}")
        try:
            # CRITIQUE : commit des offsets avant perte des partitions
            self.consumer.commit()
            logger.info("✔ Offsets commités avant rebalancing")
        except Exception as e:
            logger.error(f"Commit échoué avant rebalancing: {e}")
        finally:
            # Nettoyer les caches en mémoire de ces partitions
            for tp in revoked:
                self.offset_manager.clear_partition(tp.partition)

    def on_partitions_assigned(self, assigned):
```

on_partitions_revoked → toujours commiter les offsets en cours pour éviter de rejouer des messages déjà traités.

Performance Consumer Python — Tuning et parallélisme

Stratégies pour améliorer le débit Consumer Python

Multi-threading

Thread par partition

```
from concurrent.futures import
ThreadPoolExecutor
import threading

def process_partition(tp, messages):
    for msg in messages:
        process_order(msg.value)

with ThreadPoolExecutor(max_workers=6)
as executor:
    while running:
        msg_pack =
consumer.poll(timeout_ms=1000)
        futures =
[executor.submit(process_partition, tp,
msgs)
                    for tp, msgs in
msg_pack.items()]
        for f in futures: f.result()  #
attendre completion
        consumer.commit()  # commit
après tous les threads
```

Batch processing

Accumuler puis traiter

```
# Accumuler un batch avant traitement
BATCH_SIZE = 1000
batch = []

while running:
    msgs =
consumer.poll(timeout_ms=100)
    for tp, messages in msgs.items():
        batch.extend(messages)

    if len(batch) >= BATCH_SIZE:
        # Traitement bulk (ex: INSERT
batch en DB)
        process_bulk(batch)
        consumer.commit()
        batch = []
```

Async avec asyncio

Non-bloquant I/O

```
import asyncio
from aiokafka import AIOKafkaConsumer

async def consume():
    consumer = AIOKafkaConsumer(
        'commerce.orders',
        group_id='async-group',

bootstrap_servers='broker1:9092')
    await consumer.start()
    async for msg in consumer:
        await process_async(msg.value)
        await consumer.commit()

asyncio.run(consume())
```

Pour max perf Python : aiokafka (asyncio) ou confluent-kafka-python (librdkafka bindings C) · 3-5× plus rapide que kafka-python.

Benchmark Consumer — kafka-consumer-perf-test

⚡ kafka-consumer-perf-test.sh

Outil CLI pour mesurer le débit maximal de consommation Kafka dans votre environnement. Teste la vitesse de lecture des partitions et l'impact du zero-copy OS.

Scénarios de benchmark Consumer

```
# Benchmark débit maximal Consumer
kafka-consumer-perf-test.sh \
  --bootstrap-server localhost:19092 \
  --topic commerce.orders \
  --messages 1000000 \
  --threads 1

# Résultat attendu (3 brokers, 6 partitions) :
# start.time: 2024-01-15 09:00:00
# end.time:   2024-01-15 09:00:03
# data.consumed.in.MB: 976.56
# MB.sec: 325.52
# nMsg.sec: 341623.0
# fetch.MB.sec: 500.44
```

Mesurer le lag en temps réel

```
# Voir le lag de consommation en continu
watch -n 2 kafka-consumer-groups.sh \
  --bootstrap-server localhost:19092 \
  --describe --group orders-processor-v1
```

Résultats types Consumer

kafka-python (1 thread)
~50k msg/s

kafka-python (multi-thread)
~150k msg/s

confluent-kafka-python
~300k msg/s

Java KafkaConsumer
~500k msg/s

`fetch.min.bytes`	65536 pour gros batchs
`fetch.max.wait.ms`	500ms max attente
`max.partition.fetch.bytes`	1MB max par partition

Pour Python haute perf : confluent-kafka-python (3-5× kafka-python) · aiokafka pour I/O-bound workloads async.

Patterns Consumer avancés — Idempotence et déduplication

🔁 Idempotence côté Consumer

Kafka garantit at-least-once → un message peut être livré plusieurs fois. Le consumer DOIT être idempotent pour éviter les effets de bord (doubles insertions, double facturation, etc.).

Déduplication avec Redis

```
import redis

r = redis.Redis(host='localhost', port=6379)
DEDUP_TTL = 3600  # 1 heure

def process_idempotent(message):
    # Clé de déduplication = topic + partition + offset
    dedup_key =
f"kafka:dedup:{message.topic}:{message.partition}:{message.offset}"

    # Vérifier si déjà traité (SETNX = set if not exists)
    if r.set(dedup_key, "1", ex=DEDUP_TTL, nx=True):
        # Pas encore traité → traiter maintenant
        process_order(message.value)
        logger.info(f"✔ Traité: offset={message.offset}")
    else:
```

Déduplication avec clé métier

```
# Meilleure approche : ID métier unique dans le message
order_id = message.value.get('id')  # ex: "order-12345"
if not db.exists(f"processed_order:{order_id}"):
    process_order(message.value)
    db.mark_processed(f"processed_order:{order_id}")
```

Patterns de déduplication

Clé Kafka (offset)

Rapide, peu de code · Risque si rétention courte

Clé métier (ID)

Robuste · Survit au replay total

Redis SETNX

TTL configurable · Rapide < 1ms

Base de données

Persistant · Upsert idempotent

✔ Recommandé :
Clé métier (order_id) + DB upsert

Idempotence = si un message est rejoué, l'effet est identique. INSERT OR IGNORE, UPSERT, SETNX — patterns standards.

Quiz — Section 02 : Consumer API Python

❓ Pourquoi enable_auto_commit=False en production ?

- ○ Pour économiser des ressources
- ✔ **Auto-commit peut commiter AVANT le traitement → perte de messages si crash (at-most-once)**
- ○ Python ne supporte pas l'auto-commit
- ○ Pour améliorer les performances

❓ Que fait on_partitions_revoked() dans le RebalanceListener ?

- ○ Assigne les nouvelles partitions
- ✔ **Commite les offsets en cours AVANT que les partitions soient redistribuées**
- ○ Arrête le consumer
- ○ Envoie les messages en DLQ

❓ Qu'est-ce qu'un consumer idempotent ?

- ○ Un consumer qui ne lit qu'une fois
- ✔ **Un consumer dont le traitement peut être rejoué sans effets de bord**
- ○ Un consumer sans groupe
- ○ Un consumer avec acks=all

❓ consumer.close() est crucial car :

- ○ Il sauvegarde les messages non lus
- ✔ **Il déclenche le rebalancing immédiat et évite d'attendre session_timeout**
- ○ Il arrête Kafka
- ○ Il vide le buffer réseau

03

Sérialisation — JSON, Avro & Schema Registry

StringSerializer · JsonSerializer · Apache Avro · Schema Registry · Compatibilité

Sérialisation Kafka — Vue d'ensemble des formats

📦 **Pourquoi la sérialisation est critique**

Kafka transporte des bytes bruts. Producer sérialise objets → bytes. Consumer désérialise bytes → objets. Un mismatch Producer/Consumer = messages illisibles. Schema Registry résout ce problème.

String

`StringSerializer`

Ultra-simple. Pas de schéma. JSON textuel dans la value. Facile à débugger. Aucune évolution de schéma gérée. Dev/test uniquement.

JSON

`JsonSerializer`

Flexible. Humainement lisible. Pas de schéma imposé. Verbose (noms des champs répétés). Pas de validation native. Standard pour APIs.

Avro

`AvroSerializer`

Schéma explicite. Binaire compact (-70% vs JSON). Validation automatique. Schema Registry obligatoire. Standard production entreprise.

Protobuf

`ProtobufSerializer`

Très compact. Fortement typé. Google standard. Schema Registry Confluent. Idéal multi-langages (gRPC interop).

JSON Schema

`JsonSchemaSerializer`

JSON avec validation de schéma. Lisible + structuré. Schema Registry. Compromis Avro/JSON. Compatible REST.

Recommandation : JSON pour dev/APIs publiques · Avro pour production enterprise · Protobuf pour microservices multi-langages.

Sérialisation JSON — Java Producer + Python Consumer

Producer Java — JSON avec Jackson

```
import com.fasterxml.jackson.databind.ObjectMapper;

public class JsonProducer {
  private final ObjectMapper mapper = new ObjectMapper();

  public void sendOrder(Order order) throws Exception {
    // Sérialiser l'objet Java en JSON string
    String json = mapper.writeValueAsString(order);

    ProducerRecord<String,String> record =
      new ProducerRecord<>(
        "commerce.orders",
        order.getId(),  // clé = ID
        json);          // value = JSON string
```

Consumer Python — JSON désérialisation

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    'commerce.orders',
    group_id='python-processor',
    bootstrap_servers=['broker1:9092'],
    # Désérialisation JSON automatique
    value_deserializer=lambda m: json.loads(
        m.decode('utf-8')),
    key_deserializer=lambda m: m.decode('utf-8')
)
```

Bonnes pratiques JSON

✔ Inclure un champ 'schemaVersion'

Facilite les migrations futures sans breaking change

✔ Champs timestamp en ISO 8601

2024-01-15T09:00:00Z · universel, parseable

✔ IDs en string (pas int)

Évite les overflow et compatibilité multi-systèmes

✔ Éviter les valeurs null

Préférer omettre le champ · JSON plus compact

✔ Versionner les topics

orders.v1 → orders.v2 si breaking change

⚠️ Problème JSON sans Schema Registry

Rien n'empêche de renommer 'customerId' en 'customer_id' dans le producer
→ Consumer Python plante silencieusement

JSON est simple mais fragile sans contrat de schéma — pour la production à grande échelle, préférer Avro avec Schema Registry.

Apache Avro — Format binaire avec schéma explicite

Apache Avro

Avro est un format de sérialisation binaire avec schéma JSON. Les champs ne sont PAS inclus dans les messages (juste les valeurs) → très compact. Le schéma est stocké dans le Schema Registry et référencé par un ID numérique.

Schema Avro — order.avsc

```
{
  "type": "record",
  "name": "Order",
  "namespace": "com.acme.commerce",
  "doc": "Événement de commande e-commerce",
  "fields": [
    {"name": "id",          "type": "string",  "doc": "ID unique
commande"},
    {"name": "customerId", "type": "string",  "doc": "ID client"},
    {"name": "status",     "type": {
      "type": "enum",
      "name": "OrderStatus",
      "symbols":
["CREATED","PAID","SHIPPED","DELIVERED","CANCELLED"]
    }},
```

Producer Java avec Avro

```
props.put("value.serializer",
  "io.confluent.kafka.serializers.KafkaAvroSerializer");
props.put("schema.registry.url","http://schema-registry:8081");

Order order = new Order("order-1","cust-1",
  OrderStatus.CREATED, 99.99, System.currentTimeMillis());
```

Avro vs JSON — Comparatif

Taille message	**~70% plus petit que JSON** Binaire sans noms de champs
Performance	**2-3× plus rapide** Sér./désér. optimisée
Validation	**Automatique + stricte** Type checking au compile
Évolution	**Backward/Forward compat.** Schema Registry gère ça
Lisibilité	**Non lisible à l'œil** avro-tools pour inspecter

Avro message format : [magic byte (1)] + [schema ID (4 bytes)] + [binary avro data] = très compact.

Schema Registry — Contrat de données centralisé

🗃️ Confluent Schema Registry

Service centralisé qui stocke et versionne les schémas Avro/Protobuf/JSON Schema. Producers enregistrent un schéma → reçoivent un ID numérique. Consumers récupèrent le schéma via l'ID pour désérialiser.

Schema Registry — API REST et configuration

```
# Enregistrer un schéma manuellement
curl -X POST http://schema-registry:8081/subjects/commerce.orders-value/versions
\
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d '{"schema": "{"type":"record","name":"Order",...}"}'
# Réponse : {"id": 1}

# Lister les versions d'un schéma
curl http://schema-registry:8081/subjects/commerce.orders-value/versions
# [1, 2, 3]

# Voir le schéma version 2
curl http://schema-registry:8081/subjects/commerce.orders-value/versions/2

# Vérifier la compatibilité avant enregistrement
curl -X POST http://schema-registry:8081/compatibility/subjects/\
  commerce.orders-value/versions/latest \
  -d '{"schema": "...nouveau schéma..."}'
```

Docker Compose Schema Registry

docker-compose snippet

```
  schema-registry:
    image: confluentinc/cp-schema-registry:7.6.0
    ports: ["8081:8081"]
    environment:
      SCHEMA_REGISTRY_HOST_NAME: schema-registry

SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: >-
        broker1:9092,broker2:9092,broker3:9092
      SCHEMA_REGISTRY_LISTENERS:
http://0.0.0.0:8081
```

Modes de compatibilité des schémas

BACKWARD (défaut)

Nouveau schéma peut lire les anciens messages. Ajouter champs avec défaut OK. Supprimer champs optionnels OK.

FORWARD

Ancien schéma peut lire les nouveaux messages. Ajouter champs sans défaut. Supprimer champs avec défaut.

FULL

Backward ET Forward. La plus stricte. Ajouter/supprimer champs avec défaut seulement.

Évolution de schéma — Migration sans interruption

🔄 Évolution de schéma

Le Schema Registry permet de faire évoluer les schémas sans casser les consumers existants. Règle : ajouter des champs avec valeur par défaut, ne jamais renommer ni changer le type d'un champ existant.

✔ Modifications AUTORISÉES (BACKWARD compat.)

- ✔ Ajouter un nouveau champ avec valeur par défaut
- ✔ Supprimer un champ optionnel (avec default null)
- ✔ Ajouter un nouveau symbole à un enum

✗ Modifications INTERDITES (breaking change)

- ✗ Renommer un champ existant
- ✗ Changer le type d'un champ (string → int)
- ✗ Supprimer un champ obligatoire

Ajout champ backward-compatible

```
// Version 1 → Version 2 : ajouter 'shippingAddress'
{
  "name": "shippingAddress",
  "type": ["null", "string"],  // nullable
  "default": null,             // défaut obligatoire !
  "doc": "Adresse de livraison (ajouté v2)"
}
// Ancien consumer qui ignore ce champ → OK
// Nouveau consumer qui lit ce champ → OK
```

Stratégie breaking change : nouveau topic

```
// Si breaking change inévitable :
// 1. Créer commerce.orders.v2 (nouveau topic)
// 2. Double-écriture temporaire v1 + v2
// 3. Migrer les consumers vers v2
// 4. Arrêter l'écriture v1
// 5. Consumer groups migrent graduellement

// Config producer dual-write :
producer.send(new ProducerRecord<>("orders.v1",...));
producer.send(new ProducerRecord<>("orders.v2",...));
```

Règle d'or : toujours tester la compatibilité avec l'API SR avant de déployer un nouveau schéma en production.

Avro en Python — fastavro + Schema Registry

Producer Python avec Avro (fastavro)

```
from kafka import KafkaProducer
import fastavro, io, requests, struct

def get_schema_id(subject, schema):
    """Enregistre ou récupère l'ID du schéma"""
    url =
f"http://schema-registry:8081/subjects/{subject}/versions"
    resp = requests.post(url, json={"schema": json.dumps(schema)})
    return resp.json()["id"]

def avro_serialize(schema, data, schema_id):
    """Sérialise data en format Confluent Avro"""
    buf = io.BytesIO()
    buf.write(b'\x00')                    # magic byte
    buf.write(struct.pack('>I', schema_id)) # schema ID (4 bytes)
```

Consumer Python — Avro déserialisation

```
def avro_deserialize(raw_bytes, schema_registry_url):
    """Désérialise un message Confluent Avro"""
    magic = raw_bytes[0]
    schema_id = struct.unpack('>I', raw_bytes[1:5])[0]
    # Récupérer le schéma depuis le Registry
    schema_url = f"{schema_registry_url}/schemas/ids/{schema_id}"
    schema_json = requests.get(schema_url).json()["schema"]
    schema = fastavro.parse_schema(json.loads(schema_json))
    buf = io.BytesIO(raw_bytes[5:])
    return fastavro.schemaless_reader(buf, schema)
```

Bibliothèques Avro Python

fastavro

```
pip install fastavro
```

⭐ Recommandé — 10× plus rapide que avro-python3

avro-python3

```
pip install avro-python3
```

Officielle Apache — lente, legacy

confluent-kafka[avro]

```
pip install confluent-kafka[avro]
```

Tout-en-un + Schema Registry client intégré

confluent-kafka[avro] — plus simple

```
from confluent_kafka.avro import AvroConsumer
consumer = AvroConsumer({
  'bootstrap.servers': 'broker1:9092',
  'group.id': 'avro-group',
  'schema.registry.url': 'http://sr:8081'
})
```

Pour la production : confluent-kafka[avro] encapsule toute la gestion Schema Registry automatiquement.

Comparatif formats — Choisir la bonne sérialisation

Critère	String/JSON	Avro	Protobuf	JSON Schema
Lisibilité humaine	✔ Oui	✗ Binaire	✗ Binaire	✔ Oui
Taille message	Grande (100%)	**Petite (30%)**	**Très petite**	Grande (90%)
Performances	Moyen	⭐ **Très rapide**	⭐ **Très rapide**	Moyen
Validation schéma	✗ Manuelle	✔ **Stricte**	✔ **Stricte**	✔ **Stricte**
Schema Registry	Optionnel	Obligatoire	Optionnel	Optionnel
Évolution schéma	Manuel	✔ **Automatique**	✔ **Automatique**	Semi-auto
Usage recommandé	Dev / APIs REST	**Production enterprise**	Microservices/gRPC	APIs JSON structurées

TP Sérialisation — Exercice pratique JSON + Avro

🎯 **Objectif : Implémenter JSON et Avro sur le pipeline e-commerce**

Étape 1 — Schema Avro order.avsc

```
{
  "type": "record",
  "name": "Order",
  "namespace": "com.acme",
  "fields": [
    {"name": "id",        "type": "string"},
    {"name": "customerId","type": "string"},
    {"name": "status",    "type": {
      "type":"enum","name":"Status",
      "symbols":["CREATED","PAID","SHIPPED","DELIVERED"]
    }},
    {"name": "amount",    "type": "double"},
    {"name": "timestamp", "type": "long",
     "logicalType": "timestamp-millis"},
    {"name": "metadata",  "type": ["null",
       {"type":"map","values":"string"}],
     "default": null}
  ]
}

# Enregistrer dans Schema Registry
curl -X POST http://localhost:8081/subjects/\
  commerce.orders-value/versions \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d @order.avsc.json
```

Exercices à réaliser

1 — Créer le schéma Avro
Écrire order.avsc selon le modèle · Enregistrer dans Schema Registry · Vérifier l'ID retourné

2 — Producer JSON (Java)
Envoyer 10 commandes en JSON sur commerce.orders.json · Vérifier dans Kafka UI

3 — Producer Avro (Java)
Envoyer 10 commandes en Avro sur commerce.orders · Observer la taille des messages

4 — Consumer Python JSON
Consommer commerce.orders.json · Afficher chaque commande · Calculer le total des montants

5 — Évolution de schéma
Ajouter le champ 'shippingAddress' (nullable) · Vérifier la compatibilité backward · Déployer

Comparer la taille : JSON vs Avro sur le même message → typiquement 60-70% de réduction avec Avro.

Quiz — Section 03 : Sérialisation & Schema Registry

❓ Pourquoi Avro est plus compact que JSON ?

- ○ Avro compresse les données automatiquement
- ✔ **Les noms de champs ne sont pas inclus dans le message — seulement les valeurs**
- ○ Avro utilise des entiers plutôt que des strings
- ○ Avro supprime les espaces

❓ Compatibilité BACKWARD signifie :

- ○ Les anciens schémas peuvent lire les nouveaux messages
- ✔ **Les nouveaux schémas peuvent lire les anciens messages**
- ○ Les deux sens de compatibilité
- ○ Aucune évolution de schéma possible

❓ Format message Confluent Avro : [__ + __ + données binaires]

- ○ topic name + schema version
- ✔ **magic byte (0x00) + schema ID (4 bytes)**
- ○ producer ID + sequence number
- ○ checksum + compression type

❓ Quelle modification est AUTORISÉE en BACKWARD compat ?

- ○ Renommer le champ 'id' en 'orderId'
- ✔ **Ajouter un champ 'shippingAddress' avec default null**
- ○ Changer 'amount' de double à string
- ○ Supprimer le champ 'status'

04

Consumer Groups Avancés

Seek · Assign · Pause/Resume · Partition Assignors · Exactly-Once Consumer

Seek — Contrôle manuel de la position de lecture

📍 consumer.seek() — Repositionnement manuel

seek() permet de repositionner le consumer à n'importe quel offset d'une partition. Utile pour rejouer des événements, reprendre après un bug, tester ou implémenter des offsets dans une base externe.

Seek patterns en Java

```
// ─── SEEK TO BEGINNING ──────────────────────────────────────
consumer.subscribe(List.of("commerce.orders"));
consumer.poll(Duration.ZERO);   // force l'assignation
consumer.seekToBeginning(consumer.assignment());
// Rejouera TOUS les messages depuis le début

// ─── SEEK TO END (skip) ─────────────────────────────────────
consumer.seekToEnd(consumer.assignment());
// Ne lira que les NOUVEAUX messages à venir

// ─── SEEK TO SPECIFIC OFFSET ────────────────────────────────
TopicPartition tp = new TopicPartition("commerce.orders", 0);
consumer.seek(tp, 42);  // Lire à partir de l'offset 42

// ─── SEEK BY TIMESTAMP (replay depuis une date) ─────────────
Map<TopicPartition, Long> timestampsToSearch = new HashMap<>();
for (TopicPartition tp2 : consumer.assignment()) {
    // Rejouer depuis hier 9h00
    long yesterday9am = LocalDateTime.of(
      LocalDate.now().minusDays(1),
      LocalTime.of(9, 0))
      .toInstant(ZoneOffset.UTC).toEpochMilli();
    timestampsToSearch.put(tp2, yesterday9am);
```

Cas d'usage seek()

Replay après bug
seekToBeginning() et retraiter

Maintenance planifiée
seekToEnd() pour skipper

Audit
seek(timestamp) à date précise

Migration
seek(offset) depuis backup

Test
Rejouer des scénarios précis

Correction
Re-traiter une plage d'offsets

seekToBeginning() après un rollback applicatif = rejeu complet de l'historique des événements.

Assign vs Subscribe — Deux modes de consommation

⚖️ Subscribe vs Assign

subscribe() rejoint un Consumer Group avec rebalancing automatique. assign() assigne manuellement des partitions spécifiques — pas de group coordination, pas de rebalancing.

subscribe() — Consumer Group auto

```
// Mode standard : rebalancing automatique
consumer.subscribe(
    Arrays.asList("commerce.orders"),
    new OrdersRebalanceListener()
);
// Kafka assigne automatiquement les partitions
// Rebalancing si consumer join/leave
// Group Coordinator gère la distribution

// Cas d'usage : applications standard,
// scalabilité automatique, haute disponibilité
```

✔ Avantages

- Rebalancing automatique
- Scalabilité horizontale simple

assign() — Assignation manuelle

```
// Mode avancé : contrôle total
TopicPartition tp0 = new TopicPartition("orders", 0);
TopicPartition tp1 = new TopicPartition("orders", 1);
consumer.assign(Arrays.asList(tp0, tp1));

// Pas de Group Coordinator
// Pas de rebalancing automatique
// Seek explicite si besoin
consumer.seekToBeginning(Arrays.asList(tp0));

// Cas d'usage : outils d'admin, migration,
// replay ciblé d'une partition précise
```

✔ Avantages

- Contrôle total sur les partitions
- Pas de rebalancing → pas de stop-the-world

Pause / Resume — Contrôle de flux du consumer

⏸ consumer.pause() et consumer.resume()

pause() suspend la consommation de certaines partitions sans fermer la connexion. Le consumer continue d'envoyer des heartbeats (pas de rebalancing). resume() reprend la consommation.

Pause/Resume — Pattern backpressure

```
// Pattern : ralentir si DB surchargée (backpressure)
boolean paused = false;

while (running) {
    ConsumerRecords<String,String> records =
        consumer.poll(Duration.ofMillis(100));

    // Vérifier la charge de la DB cible
    if (!paused && database.isOverloaded()) {
        // Mettre en pause toutes les partitions
        consumer.pause(consumer.assignment());
        paused = true;
        log.warn("Consumer mis en pause — DB surchargée");
    }

    if (paused && !database.isOverloaded()) {
        // Reprendre quand DB est OK
        consumer.resume(consumer.assignment());
        paused = false;
        log.info("Consumer repris — DB OK");
    }

    if (!paused) {
        for (ConsumerRecord<String,String> r : records) {
```

Cas d'usage pause/resume

Backpressure
DB ou API downstream surchargée

Rate limiting
Respecter les quotas d'une API externe

Maintenance
Drain gracieux avant redémarrage

Priorité
Pauser les partitions basse priorité

Circuit breaker
Pause si downstream circuit ouvert

✔ Avantage clé : heartbeat maintenu
→ pas de rebalancing pendant la pause

Pendant la pause, le lag Kafka augmente mais les messages sont conservés — reprise propre quand resume().

Partition Assignors — Algorithmes de distribution

⚙️ Partition Assignment Strategies

Le Partition Assignor détermine comment les partitions sont distribuées entre les consumers d'un groupe lors d'un rebalancing. Kafka 3.x propose 4 stratégies natives.

RangeAssignor (défaut)

Partitions consécutives par consumer · Simple mais déséquilibré si nb partitions % nb consumers ≠ 0

```
// Exemple : 6 partitions, 2 consumers
// Consumer 1 : P0, P1, P2
// Consumer 2 : P3, P4, P5
//
```

```
// Problème : si 7 partitions, 2
consumers :
// C1 : P0, P1, P2, P3 (4 partitions)
// C2 : P4, P5, P6     (3 partitions)
// → Déséquilibre !
```

RoundRobinAssignor

Distribution circulaire · Meilleur équilibre sur plusieurs topics

```
// Exemple : 6 partitions, 2 consumers
// C1 : P0, P2, P4 (alternées)
// C2 : P1, P3, P5 (alternées)
//
```

```
// Multi-topics :
// C1 : orders-P0, events-P0, events-P2
// C2 : orders-P1, events-P1, events-P3
// → Équilibré sur tous les topics !
```

StickyAssignor

Minimise les mouvements lors du rebalancing · Garde les assignations actuelles autant que possible

```
// Consumer 3 quitte le groupe
// BEFORE: C1:P0,P1 · C2:P2,P3 ·
C3:P4,P5
// Range/RR: tout réassigne depuis zéro
// Sticky: C1:P0,P1,P4 · C2:P2,P3,P5
// → Seules P4 et P5 ont bougé !
// Avantage : moins d'interruptions,
// caches locaux préservés
```

Config : partition.assignment.strategy=org.apache.kafka.clients.consumer.CooperativeStickyAssignor (Kafka 2.4+ recommandé)

CooperativeStickyAssignor — Rebalancing sans interruption

🔄 **CooperativeStickyAssignor (Kafka 2.4+)**

Révolutionne le rebalancing : au lieu de révoquer TOUTES les partitions (stop-the-world), seules les partitions qui changent de propriétaire sont révoquées. Les autres continuent de consommer pendant le rebalancing !

Rebalancing Eager (classique)

Stop the world complet

1. TOUS les consumers stoppent
2. TOUTES les partitions révoquées
3. Redistribution complète
4. TOUS reprennent

→ Interruption de 1-30 secondes
→ Impact fort sur le débit

Protocoles : RangeAssignor, RoundRobinAssignor, StickyAssignor

Rebalancing Coopératif (Kafka 2.4+)

Incrémental — sans stop the world

1. Seules les partitions concernées sont révoquées (minority)
2. Les autres continuent de tourner
3. Redistribution partielle uniquement
4. Deuxième round si besoin

→ Interruption quasi-nulle
→ Débit maintenu à 90%+

Protocole : CooperativeStickyAssignor

Config : partition.assignment.strategy=CooperativeStickyAssignor · Compatible Kafka 2.4+ · Migration rolling possible.

Exactly-Once Consumer — Transactions Kafka

🎯 Exactly-Once Processing (EOS)

EOS combine producer idempotent + transactions pour garantir qu'un message est traité exactement une fois dans un pipeline read-process-write. isolation.level=read_committed côté consumer.

Exactly-Once — Read-Process-Write Java

```
// Consumer avec isolation read_committed
props.put("isolation.level", "read_committed");
KafkaConsumer<String,String> consumer = new KafkaConsumer<>(props);

// Producer transactionnel
props2.put("transactional.id","orders-transformer-1");
KafkaProducer<String,String> producer = new KafkaProducer<>(props2);
producer.initTransactions();

while (running) {
    ConsumerRecords<String,String> records =
        consumer.poll(Duration.ofMillis(100));

    producer.beginTransaction();
    try {
        for (ConsumerRecord<String,String> record : records) {
            // Transformer et écrire dans un autre topic
            String enriched = enrich(record.value());
            producer.send(new ProducerRecord<>(
                "commerce.orders.enriched",
                record.key(), enriched));
        }
        // Commiter les offsets DANS la transaction
        // = atomique avec l'écriture
```

Garanties EOS

Idempotence Producer

Pas de doublons broker-side

`enable.idempotence=true`

Transactions

Atomique multi-topics

`transactional.id configuré`

read_committed

Consumer ne lit que msgs commités

`isolation.level=read_committed`

sendOffsetsToTransaction

Commit offset dans la transaction

`Atomique avec l'écriture`

EOS a un coût en débit (~20-30%). Utiliser uniquement pour les données critiques : finance, facturation, CDC.

Quiz — Section 04 : Consumer Groups avancés

❓ consumer.pause() permet de :

- ○ Arrêter définitivement le consumer
- ✔ **Suspendre la consommation en maintenant le heartbeat (pas de rebalancing)**
- ○ Mettre en pause Kafka
- ○ Commiter les offsets actuels

❓ Différence seek() vs subscribe() ?

- ○ seek() est plus rapide
- ✔ **subscribe() = groupe auto rebalancing · seek() = repositionnement offset manuel**
- ○ Ils font la même chose
- ○ seek() ne fonctionne qu'avec assign()

❓ CooperativeStickyAssignor améliore le rebalancing car :

- ○ Il est plus rapide que les autres
- ✔ **Seules les partitions affectées sont révoquées — les autres continuent**
- ○ Il supprime le rebalancing
- ○ Il nécessite moins de brokers

❓ isolation.level=read_committed signifie :

- ○ Messages chiffrés en transit
- ✔ **Consumer ne lit que les messages commités dans les transactions (EOS)**
- ○ Consumer commite ses offsets automatiquement
- ○ Lecture séquentielle obligatoire

05

TP — Pipeline e-commerce Java & Python

Producer Java · Consumer Python · Avro · DLQ · Consumer Groups · Exactly-Once

Architecture TP — Pipeline e-commerce complet

🏗️ **Pipeline fil rouge Jour 2**

Construction d'un pipeline e-commerce complet : Producer Java émet les événements commandes, Consumer Python les traite, agrège et persiste. Avro pour la sérialisation, DLQ pour les erreurs, Consumer Groups pour le scaling.

Order API (Java Spring)
Simule les commandes e-com

Schema Registry
Schémas Avro validation auto

Kafka Cluster 3 brokers KRaft
Topics orders events · dlq

Kafka UI
Monitoring visualisation

Order Processor (Python)
Traitement + DB insert

Analytics (Python)
Agrégation CA / metrics

Error Handler (Python)
Rejoue DLQ monitoring

Topics Kafka du TP

`commerce.orders`
6P RF=3 · Avro · Événements commandes

`commerce.orders.dlq`
1P RF=2 · JSON · Messages en erreur

`analytics.orders.summary`
3P RF=3 · JSON · Agrégats par heure

EX 1 — Producer Java : Événements commandes (30 min)

🎯 **Objectif : Implémenter un OrderProducer Java avec Avro, idempotence et callbacks**

OrderProducer.java — Squelette à compléter

```
@Component
public class OrderProducer {

    @Autowired
    private KafkaTemplate<String, Order> kafkaTemplate;

    // TODO 1 : Configurer acks=all + enable.idempotence=true
    // TODO 2 : Ajouter compression snappy + batch.size=65536

    public void sendOrder(Order order) {
        // TODO 3 : Créer un ProducerRecord avec la clé order.getId()
        ProducerRecord<String, Order> record = /* ??? */;

        // TODO 4 : Envoyer avec callback (log offset + erreur DLQ)
        kafkaTemplate.send(record)
            .addCallback(
                success -> log.info("Sent: offset={}",
                    success.getRecordMetadata().offset()),
                failure -> {
                    // TODO 5 : Envoyer en DLQ si erreur permanente
                    log.error("Failed: {}", failure.getMessage());
                }
            );
    }
```

Checklist de validation

- ☐ 100 commandes envoyées en Avro
- ☐ Kafka UI → topic commerce.orders
- ☐ 6 partitions avec messages répartis
- ☐ Log : offsets confirmés dans le callback
- ☐ DLQ : test avec message malformé
- ☐ Vérifier acks=all dans les métriques JMX
- ☐ Mesurer le débit : msgs/sec dans UI

Squelette complet : kafka-formation/jour2/src/main/java/com/acme/OrderProducer.java

EX 2 — Consumer Python : Order Processor (30 min)

🎯 **Objectif : Consumer Python avec commit manuel, DLQ, RebalanceListener et métriques**

order_processor.py — Squelette à compléter

```
from kafka import KafkaConsumer, KafkaProducer
from kafka.errors import CommitFailedError
import json, logging, time

# TODO 1 : Créer le KafkaConsumer
# - group_id = 'order-processor-v1'
# - enable_auto_commit = False
# - value_deserializer : désérialiser JSON
# - max_poll_records = 500
consumer = # ???

# TODO 2 : Créer le KafkaProducer pour la DLQ
dlq_producer = # ???

# TODO 3 : Implémenter le RebalanceListener
# - on_partitions_revoked → commitSync()
# - on_partitions_assigned → log les partitions
listener = # ???

# TODO 4 : Poll loop avec gestion d'erreurs
try:
    while running:
        msg_pack = consumer.poll(timeout_ms=1000)
        for tp, messages in msg_pack.items():
            for message in messages:
```

Checklist de validation

- ☐ Consumer démarre et lit les messages
- ☐ Logs : traitement de chaque commande
- ☐ Commit manuel après chaque batch
- ☐ Messages invalides → DLQ
- ☐ RebalanceListener logge les partitions
- ☐ kafka-consumer-groups LAG = 0
- ☐ Lancer 3 instances → voir rebalancing
- ☐ BONUS : implement idempotent processing

EX 3 — Sérialisation Avro complète (25 min)

🎯 **Objectif : Schema Registry + Avro Producer Java + Consumer Python + évolution de schéma**

1 **Enregistrer le schéma**

Lancer Schema Registry · Enregistrer order.avsc · Vérifier l'ID (curl http://localhost:8081/subjects)

2 **Producer Avro Java**

Configurer KafkaAvroSerializer · Envoyer 50 commandes · Observer dans Kafka UI (format Avro)

3 **Consumer Avro Python**

pip install fastavro · Implémenter avro_deserialize · Consommer et afficher les commandes

4 **Comparer tailles JSON vs Avro**

Produire 100 msgs JSON et 100 msgs Avro · Comparer dans Kafka UI · Calculer le ratio de réduction

5 **Évolution du schéma**

Ajouter champ 'shippingAddress' (nullable) · Tester compatibilité backward · Déployer v2 sans arrêt

Résultat attendu : ~65-70% de réduction de taille JSON → Avro sur les messages commandes.

EX 4 — Consumer Groups : Scaling et résilience (20 min)

🎯 **Objectif : Observer le scaling horizontal et la résilience des Consumer Groups**

Séquence à suivre

```
# SETUP : S'assurer que le topic a 6 partitions
kafka-topics.sh --describe --topic commerce.orders
# → 6 partitions

# ─── Étape 1 : 1 consumer (sous-optimal) ──────────────────
python order_processor.py --group=perf-test &
# Observer : kafka-consumer-groups.sh → 1 consumer, 6 partitions

# ─── Étape 2 : 3 consumers (mi-optimal) ───────────────────
python order_processor.py --group=perf-test &
python order_processor.py --group=perf-test &
# Observer le rebalancing dans les logs
# kafka-consumer-groups.sh → 3 consumers, 2 partitions chacun

# ─── Étape 3 : 6 consumers (optimal) ──────────────────────
python order_processor.py --group=perf-test &
python order_processor.py --group=perf-test &
python order_processor.py --group=perf-test &
# kafka-consumer-groups.sh → 6 consumers, 1 partition chacun
# Mesurer le LAG → doit descendre plus vite

# ─── Étape 4 : 7 consumers (surplus) ──────────────────────
python order_processor.py --group=perf-test &
# Le 7ème consumer est IDLE (0 partition assignée)
```

Points à mesurer

1 consumer
LAG croît si msgs vite

3 consumers
LAG 3× plus faible

6 consumers
LAG minimal — optimal

7 consumers
Consumer 7 = IDLE

Panne 1 consumer
Rebalancing < 30s

Reprise
LAG se résorbe

Règle d'or vérifiée : nb consumers actifs = min(nb lancés, nb partitions). Le 7ème est IDLE.

EX 5 — Analytics Consumer Python (20 min)

🎯 **Objectif : Consumer Python qui agrège les commandes et calcule le CA en temps réel**

analytics_consumer.py — À compléter

```
from collections import defaultdict
from kafka import KafkaConsumer
import json

# Métriques en mémoire (simple pour le TP)
stats = defaultdict(lambda: {"count": 0, "total": 0.0})

consumer = KafkaConsumer(
    'commerce.orders',
    group_id='analytics-processor',
    bootstrap_servers=['broker1:9092'],
    auto_offset_reset='earliest',
    enable_auto_commit=False,
    value_deserializer=lambda m: json.loads(m.decode())
)

print("Analytics consumer démarré...")
for message in consumer:
    order = message.value

    # TODO 1 : Agréger par status
    status = order.get('status', 'UNKNOWN')
    amount = order.get('amount', 0.0)
    stats[status]['count'] += 1
    stats[status]['total'] += amount
```

Résultat attendu

Dashboard en temps réel

```
=== ANALYTICS DASHBOARD ===
  CANCELLED    :    15 msgs |
               CA:   1250.50 EUR
  CREATED      :   245 msgs |
               CA:  24350.75 EUR
  DELIVERED    :    80 msgs |
               CA:   8200.00 EUR
  PAID         :   200 msgs |
               CA:  19875.25 EUR
  SHIPPED      :   160 msgs |
               CA:  15640.00 EUR
  TOTAL : 700 messages traités
```

✔ Commit tous les 100 msgs : Évite OOM si milliers de msgs

✔ defaultdict : Initialisation auto des compteurs

✔ BONUS : Écrire sur summary : Pipeline complet Producer/Consumer

BONUS : Écrire les agrégats sur analytics.orders.summary puis lire avec un 3ème consumer — pipeline complet !

EX 6 — Exactly-Once : Pipeline transactionnel (25 min)

🎯 **Objectif : Implémenter le pattern Read-Process-Write avec transactions Kafka**

OrderEnricher.java — Exactly-Once Read→Write

```
@Component
public class OrderEnricher {

    // TODO 1 : Configurer le consumer avec
    // isolation.level = read_committed

    // TODO 2 : Configurer le producer avec
    // transactional.id = "order-enricher-1"
    // enable.idempotence = true

    public void run() {
        producer.initTransactions();

        while (running) {
            var records = consumer.poll(Duration.ofMillis(100));
            producer.beginTransaction();
            try {
                for (var record : records.records("commerce.orders")) {
```

Validation EOS

- ☐ isolation.level=read_committed
- ☐ transactional.id configuré
- ☐ initTransactions() appelé
- ☐ beginTransaction() avant poll
- ☐ sendOffsetsToTransaction() atomique
- ☐ commitTransaction() après tout
- ☐ abortTransaction() en cas d'erreur

Test idempotence : tuer l'enricher pendant une transaction

```
# Lancer l'enricher · Tuer avec kill -9 pendant une transaction
# Résultat attendu : message pas dupliqué dans commerce.orders.enriched
# Kafka rejoue la transaction non commité · Exactement une fois !
```

EOS test : kill -9 pendant une transaction → Kafka doit rejouer sans dupliquer les messages enrichis.

Debugging Kafka — Outils et commandes indispensables

Boîte à outils du développeur Kafka

Commandes de diagnostic essentielles

```
# ─── Voir le LAG d'un groupe ──────────────────────────
kafka-consumer-groups.sh \
  --bootstrap-server localhost:19092 \
  --describe --group order-processor-v1

# ─── Voir les messages d'une partition ────────────────
kafka-console-consumer.sh \
  --bootstrap-server localhost:19092 \
  --topic commerce.orders \
  --partition 0 --offset 0 --max-messages 10 \
  --property print.offset=true \
  --property print.key=true

# ─── Compter les messages d'un topic ──────────────────
kafka-run-class.sh kafka.tools.GetOffsetShell \
```

Python — Debugging du consumer

```
# Voir où en est le consumer
from kafka import KafkaConsumer, TopicPartition

consumer = KafkaConsumer(group_id='debug', ...)
consumer.subscribe(['commerce.orders'])
consumer.poll(0)  # force assignation

# Position actuelle
for tp in consumer.assignment():
    pos = consumer.position(tp)
    committed = consumer.committed(tp)
    end = consumer.end_offsets([tp])[tp]
    lag = end - (committed.offset if committed else 0)
    print(f"P{tp.partition}: pos={pos}"
          f" committed={committed} lag={lag}")
```

Kafka UI — Fonctionnalités clés

```
# Kafka UI : http://localhost:8080
# — Dashboard : brokers, topics, consumers
# — Browse messages par partition/offset
# — Consumer Groups : lag en temps réel
# — Schema Registry : schémas enregistrés
# — Produce : envoyer messages de test
# — Topics : configs, taille, rétention
```

Erreurs courantes et solutions

```
# WARN: Heartbeat missed → max.poll.interval.ms trop court
# FIX: augmenter max.poll.interval.ms ou réduire batch

# ERROR: CommitFailedError after rebalancing
# FIX: toujours commiter dans on_partitions_revoked()

# WARN: Producer blocked on full buffer
# FIX: augmenter buffer.memory ou réduire max.block.ms
```

Kafka UI est l'outil de debug #1 — toujours l'avoir ouvert pendant le développement.

Récapitulatif Jour 2 — 6 compétences acquises + Aperçu J3

☕ Producer API Java

KafkaProducer · acks · idempotence · transactions · callbacks · DLQ · compression snappy

🎯 Exactly-Once

Idempotence · transactions · isolation.level · sendOffsetsToTransaction · atomicité

🐍 Consumer API Python

KafkaConsumer · poll · commit manuel · RebalanceListener · error handling · DLQ · async

🔄 Consumer Groups avancés

Seek · assign · pause/resume · CooperativeStickyAssignor · scaling horizontal

📦 Sérialisation

JSON simple · Avro binaire compact · Schema Registry · compatibilité backward/forward

🏗️ Pipeline complet

Architecture Producer Java ⟶ Kafka ⟶ Consumer Python · Analytics · Debugging